

Design, Train, Monitor, and Deploy a scalable
Cloud-Based Machine Learning System that
integrates explainable Deep Reinforcement
Learning algorithms, interpreted through SHAP,
to signal optimized decisions based on real, live
data market to augment human decisions

Julia Beijia Jiang Wake Forest University URECA Scholarship Project

June 2025 - September 2025

1 Data Pipeline

As experienced machine learning engineers consistently emphasize, machine learning models are driven as equally by data quality as algorithmic sophistication. In financial markets, this principle becomes even more critical due to the dynamic, noisy, and non-stationary nature of market data. High-quality datasets are fundamental for model performance, particularly when developing reinforcement learning agents that must adapt to rapidly changing market conditions. Through **systematic collection with integrated API, data preprocessing, and data engineering of relevant technical indicators**, I aim to construct a robust, comprehensive data pipeline representative of financial market, specifically designed for training and evaluating the proposed deep reinforcement learning system for portfolio management.

1.1 Data Availability and Collection

I selected **yfinance** as my primary data source, demonstrating practical API selection for financial data acquisition. This Python wrapper for Yahoo Finance provides over 20 years of comprehensive market data with reliable access and cost-effectiveness (completely free), making it suitable for research and development projects. My selection was strategically driven by its built-in **auto_adjust** functionality, which addresses a critical data quality challenge that many analysts overlook. This feature automatically corrects for corporate actions (stock splits, dividends, spin-offs) that create false signals in raw price data—without this preprocessing, a 2-for-1 stock split appears as a 50% overnight loss, corrupting model training and backtesting results.

```

Columns immediately after extraction: ['Adj Close', 'Close', 'High', 'Low', 'Open', 'Volume', 'Stock']
Sample data:
Adj Close      Close      High      Low      Open      Volume \
Date
2009-01-02  29.900190  49.489967  49.657192  48.093643  48.118729  6355424
2009-01-05  29.551636  48.913044  49.406357  48.586956  49.155518  4507485
2009-01-06  29.900190  49.489967  50.008362  49.005016  49.113712  5939575
2009-01-07  29.334408  48.553513  49.297661  48.143814  49.005016  5499328
2009-01-08  29.592039  48.979935  49.088631  48.035118  48.227425  4718100

Stock
Date
2009-01-02  MMM
2009-01-05  MMM
2009-01-06  MMM
2009-01-07  MMM
2009-01-08  MMM

```

Figure 1: Dataset Before Auto Adjusted Price

```

Columns immediately after extraction: ['Close', 'High', 'Low', 'Open', 'Volume', 'Stock']
Sample data:
Close      High      Low      Open      Volume Stock
Date
2009-01-02  29.900190  30.001222  29.056578  29.071734  6355424  MMM
2009-01-05  29.551636  29.849679  29.354624  29.698130  4507485  MMM
2009-01-06  29.900190  30.213387  29.607199  29.672870  5939575  MMM
2009-01-07  29.334408  29.783997  29.086882  29.607191  5499328  MMM
2009-01-08  29.592039  29.657710  29.021213  29.137398  4718100  MMM

```

Figure 2: Dataset After Auto Adjusted Price

For the **Open High Low Close Volume** data collection, I selected the **Dow Jones Industrial Average 30 stocks** as my training/testing data source, balancing system complexity with computational possibility. While the S&P 500 would provide broader market representation covering approximately 80% of US market capitalization versus the Dow’s 25%, the focused 30-stock market offers practical advantages for proof-of-concept development: blue-chip liquidity, **improved computational complexity**, consistent data quality, and **at least 16x lower computational overhead for faster model iteration and hyperparameter optimization**. The Dow’s sector diversity, while limited, provides sufficient complexity for demonstrating reinforcement learning capabilities in cross-asset relationships and portfolio optimization, with future expansion to the S&P 500 representing a natural scalability enhancement for production deployment.

1.2 Data Formatting, Data Preprocessing and Data Engineering

The *data formatting pipeline* implemented comprehensive structural transformations to optimize the dataset for machine learning consumption. **The process systematically drops missing data by rows, resets the index from date-ordered timestamps to integer indices labeled 'Index' for consistent numerical referencing**, and reorders columns by Date followed by Stock Name in ascending format to ensure predictable data organization.

Stock Date	CAT	CRM	CSCO	CVX	DIS
2000-01-03	12.610167	NaN	35.360302	16.231291	22.736225
2000-01-04	12.448087	NaN	33.376522	16.231291	24.068054
2000-01-05	12.739840	NaN	33.274281	16.522421	25.066927
2000-01-06	13.388186	NaN	32.722092	17.226027	24.068054
2000-01-07	13.825808	NaN	34.644516	17.529301	23.687531
...	...	...	...	...	...
2025-07-25	433.750000	269.109985	68.690002	154.839996	121.470001
2025-07-28	432.940002	270.250000	67.919998	156.279999	120.370003
2025-07-29	430.049988	267.390015	67.959999	157.029999	119.910004
2025-07-30	434.119995	264.809998	68.279999	153.559998	119.540001
2025-07-31	438.019989	258.329987	68.080002	151.639999	119.110001

Figure 3: NaN rows in Stock *CRM*

Additionally, the pipeline **reformats the index structure, creates an additional 'Day' column to capture day-of-week effects**—a well-documented phenomenon in financial markets where certain weekdays exhibit distinct trading patterns—and completely drops the old temporal index to eliminate redundancy. *This formatting strategy transforms the raw financial data into a standardized, machine-readable format while preserving critical temporal relationships and market microstructure effects essential for reinforcement learning agent training.*

The **data engineering phase** encompassed comprehensive feature engineering beyond basic price data, implementing industry-standard technical indicators to provide comprehensive market information for the deep reinforcement learning agents. The technical indicator selection process balanced different time horizons, signal types, and complementary relationships, with criteria based on common trading platform practices representing over 80% usage in quantitative finance. Selected indicators included MACD for momentum and trend identification, Bollinger Bands (boll_ub, boll_lb) for volatility assessment, Simple Moving Averages (close_30_sma, close_60_sma) for trend following, multiple RSI periods (rsi_14, rsi_30) for overbought/oversold conditions, CCI_30 for cyclical trend detection, and dx_30 for trend strength measurement. This selection, inspired by FinRL library standards with additional rsi_14 for optimized industry alignment, ensures realistic learning conditions for reinforcement learning agents.

The engineering process utilized the StockDataFrame library to systematically calculate indicators across all 28 stocks, with careful handling of missing values that concentrated in the dataset's initial trading days (January 3-4, 2000). Rather than imputing these NaN values with potentially misleading zeros, the dataset was truncated to begin training from January 5, 2000, ensuring mathematical validity across all technical indicators.

Additionally, the system integrated the VIX volatility index and engineered a custom turbulence index using a 252-day rolling covariance matrix to measure market regime shifts, providing agents with sophisticated risk assessment capabilities during volatile market conditions.

More significantly, technical indicator calculations introduced NaN values in six out of nine indicators (boll_ub, boll_lb, rsi_14, rsi_30, cci_30, dx_30) with counts ranging from 28 to 30 missing values.

Stock Date	CAT	CSCO	CVX	DIS	GS
2000-01-03	12.610167	35.360302	16.231291	22.736225	61.993763
2000-01-04	12.448087	33.376522	16.231291	24.068054	58.088970
2000-01-05	12.739840	33.274281	16.522421	25.066927	55.368809
2000-01-06	13.388186	32.722092	17.226827	24.068054	57.737995
2000-01-07	13.825088	34.644516	17.529301	23.687531	57.957359
...	...	...	...	...	...
2025-07-25	433.750000	68.690002	154.839996	121.470001	728.979980
2025-07-28	432.940002	67.919998	156.279999	120.370003	723.650024
2025-07-29	430.049988	67.959999	157.029999	119.910004	731.979980
2025-07-30	434.119995	68.279999	153.559998	119.540001	730.750000
2025-07-31	438.019989	68.080002	151.639999	119.110001	723.590027

Figure 4: stock *CRM* is removed completely

Detailed temporal analysis revealed these NaN values concentrated exclusively in the dataset’s initial trading days (January 3-4, 2000), occurring due to insufficient historical data for technical indicator calculations. Rather than imputing these values with potentially misleading zeros—which could signal false trading conditions to the reinforcement learning algorithm—I implemented a systematic solution by truncating the dataset to begin training from January 5, 2000, ensuring all technical indicators maintain mathematical validity throughout the training period.

1.3 Data Representation and Feature Importance Analysis

The comprehensive data representation analysis revealed critical insights into feature distributions and their relevance for deep reinforcement learning applications. Distribution analysis across all engineered features demonstrated appropriate statistical properties for model training, with close prices exhibiting the expected right-skewed distribution typical of financial assets, while volume data showed normal distribution characteristics when log-transformed.

Technical indicators displayed theoretically sound distributions, with **RSI-30 values** appropriately bounded within 0-100 range and showing clear over-sold/overbought regions, **MACD distributions** centered around zero with distinct positive and negative momentum signals, and **CCI-30 values** exhibiting the expected oscillatory behavior around standard deviation thresholds. Market volatility measures proved particularly informative, with **VIX distributions** clearly delineating low volatility ($VIX \leq 20$) and high volatility ($VIX \geq 30$) regimes, occurring on 3,217 and 534 trading days respectively over the 20-year period, while the **custom turbulence index** showed heavily right-skewed distribution indicating rare but significant market stress events. **Feature correlation matrix analysis** revealed low to moderate correlations between most technical indicators (ranging from -0.34 to 0.80), confirming the complementary nature of the selected feature set and reducing multicollinearity concerns for the reinforcement learning agent.

Individual stock trajectory analysis across all 28 Dow components demonstrated *consistent long-term upward trends with varying volatility patterns*, while the equal-weighted portfolio achieved a total return of 419.9% from 2000-2020 with price appreciation from \$14.65 to \$130.56, validating the

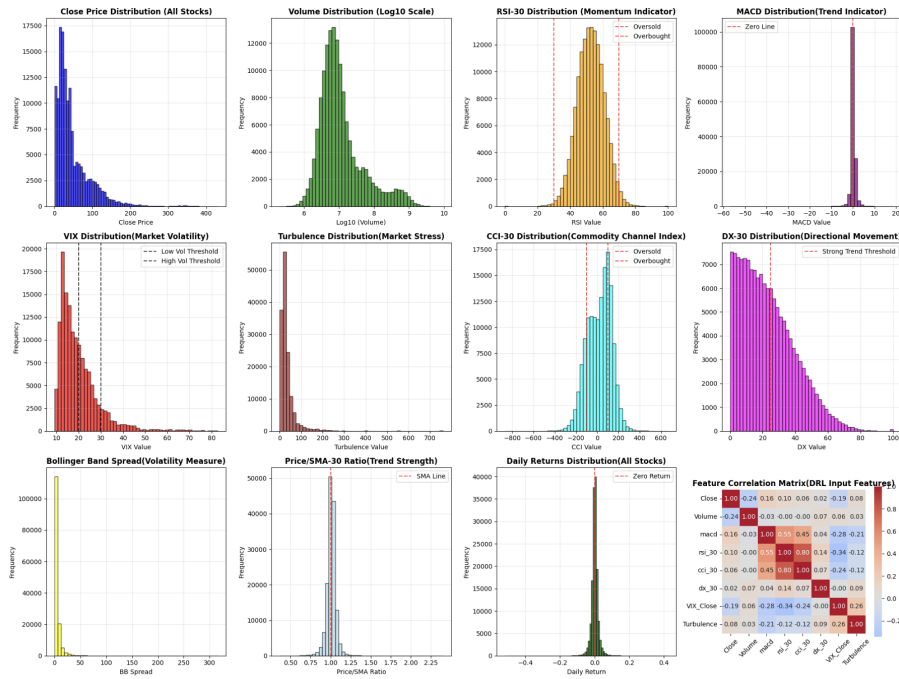


Figure 5: Feature Importance and Technical Indicator Distributions

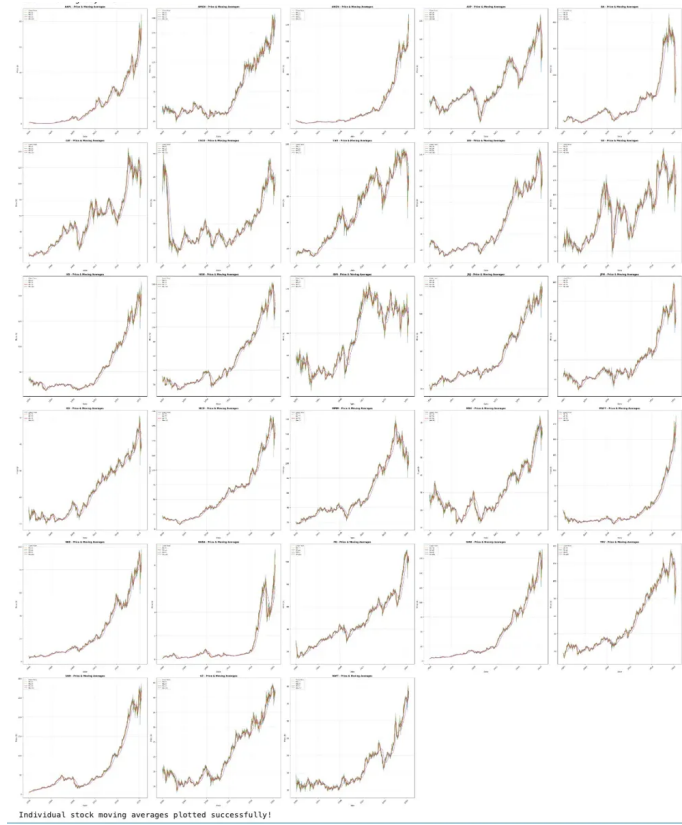


Figure 6: Moving Averages Trend and Price of Each Individual Stock

dataset's representation of realistic market conditions across multiple economic cycles including the dot-com crash, 2008 financial crisis, and various market corrections that provide diverse learning scenarios for the deep reinforcement learning system.

1.4 Data Storage and Optimization

The data storage strategy implemented a dual-format approach optimizing for both development efficiency and cost management, **utilizing both CSV and Apache Parquet formats** for different temporal ranges and operational requirements.

The system maintains two primary datasets: a CSV file containing data from 2009-2020 and a Parquet training dataset spanning from 2000-01-01 to 2020-06-19, providing broader historical context for model training.

The Parquet format provided significant advantages for the financial time series dataset through its **columnar storage structure enabling efficient**

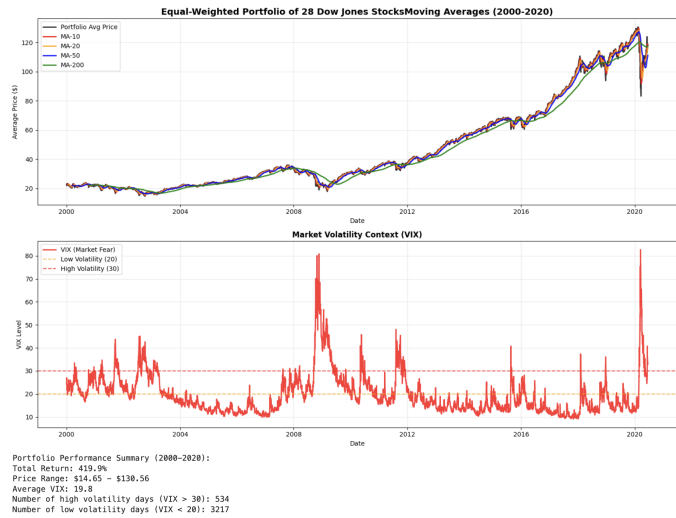


Figure 7: Enter Caption

```

Index: 144144 entries, 0 to 5147
Data columns (total 19 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Date                 144144 non-null  datetime64[ns]
1   Close                144144 non-null  float64
2   High                 144144 non-null  float64
3   Low                  144144 non-null  float64
4   Open                 144144 non-null  float64
5   Volume               144144 non-null  int64
6   Stock                144144 non-null  object
7   Day                  144144 non-null  int32
8   macd                 144144 non-null  float64
9   boll_ub              144116 non-null  float64
10  boll_lb              144116 non-null  float64
11  rsi_14               144115 non-null  float64
12  rsi_30               144115 non-null  float64
13  cci_30               144116 non-null  float64
14  dx_30                144114 non-null  float64
15  close_30_sma         144144 non-null  float64
16  close_60_sma         144144 non-null  float64
17  VIX_Close            144144 non-null  float64
18  Turbulence           144144 non-null  float64
dtypes: datetime64[ns](1), float64(15), int32(1), int64(1), object(1)
memory usage: 21.4+ MB
Current training datasize takes 19201031 bytes and 18.311530113220215 MegaBytes in memory.
Current testing datasize takes 4777381 bytes and 4.556065559387207 MegaBytes in memory.

```

Figure 8: Parquet DataSize

compression, optimized read performance for analytical queries commonly required in reinforcement learning training loops. Additionally, **Parquet’s support for predicate pushdown and column pruning capabilities** proved crucial for the deep reinforcement learning system, as training routines often require selective feature access rather than full dataset scanning, while its native integration with modern data processing frameworks aligned with the project’s cloud-based deployment objectives. **This strategic separation allows for comprehensive backtesting capabilities while maintaining computational efficiency during development iterations.**

Both final training and testing datasets were extracted and engineered into CSV/Parquet files stored locally, significantly reducing storage costs on AWS cloud services—a critical consideration for scalable machine learning operations. **The optimized storage footprint achieved remarkable efficiency with the longer training dataset consuming only 18.31 MB and the testing dataset requiring 4.56 MB, totaling just 22.87 MB of storage space.** Despite the compact size, data sufficiency analysis confirmed the **training dataset’s adequacy for neural network training, containing 83,897/144,444 structured data rows that undergo 10^7 timestep training iterations,** ensuring the deep reinforcement learning models experience sufficient data exposure for meaningful pattern recognition while establishing a foundation for efficient scaling when expanding to larger datasets like the S&P 500 in future system enhancements.